

Herding

Nguyễn Quang Dũng

Ngày 4 tháng 3 năm 2025

Tóm tắt nội dung

Herding là một thuật toán nổi bật trong các phương pháp chọn coresets dựa trên tiếp cận hình học. Nó được sử dụng rộng rãi trong các bài toán tối ưu hóa, giảm kích thước dữ liệu và ước lượng mật độ kernel.

1 Introduction

Phát biểu bài toán

- **Input:** Tập dữ liệu $\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$, trong đó mỗi $\mathbf{x}_i \in \mathbb{R}^d$ là một điểm dữ liệu và số lượng điểm T cần chọn để tạo thành coreset.
- **Output:** Một tập hợp coreset $\mathcal{C} = \{\mathbf{x}_{t_1}, \mathbf{x}_{t_2}, \dots, \mathbf{x}_{t_T}\}$, chứa T điểm từ $\mathcal{X}$, đại diện tốt nhất cho toàn bộ dữ liệu.

Cách hoạt động của Herding

- Bước 1 (Khởi tạo): Bắt đầu với vector trọng số ban đầu: $\mathbf{w}_0 = \mathbf{0}$ (vector không có ảnh hưởng ban đầu).
- Bước 2: (Lặp) Với mỗi bước lặp $t = 1, 2, \dots, T$, thực hiện hai thao tác chính:

– Chọn điểm quan trọng nhất:

- * Điểm $\mathbf{x}_t$ được chọn từ tập $\mathcal{X}$ sao cho tối ưu hóa hàm mục tiêu:

$$\mathbf{x}_t = \arg \max_{\mathbf{x} \in \mathcal{X}} \langle \mathbf{x}, \mathbf{w}_t \rangle.$$

- * Trong đó $\langle \mathbf{x}, \mathbf{w}_t \rangle$: tích vô hướng giữa $\mathbf{x}$ và trọng số hiện tại $\mathbf{w}_t$, phản ánh mức độ ảnh hưởng của $\mathbf{x}$ tại thời điểm hiện tại.

– Cập nhật vector trọng số:

- * Sau khi chọn $\mathbf{x}_t$, trọng số $\mathbf{w}_t$ được cập nhật theo công thức:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \mathbf{x}_t - \bar{\mathbf{x}}.$$

* Trong đó: $\bar{\mathbf{x}} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i$: giá trị trung bình của toàn bộ dữ liệu và việc trừ $\bar{\mathbf{x}}$ giúp cân bằng trọng số để tránh chọn quá nhiều điểm từ các khu vực đã được đại diện.

- Bước 3: (Kết quả)

- Sau T bước lặp, tập hợp $\mathcal{C} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T\}$ chính là coreset được chọn.
- Tập $\mathcal{C}$ đại diện tốt cho toàn bộ dữ liệu $\mathcal{X}$, bảo toàn các đặc điểm quan trọng về phân phối dữ liệu.

Thực tế trong bài báo DeepCore

Bước 1 (Khởi tạo)

Tính trung bình của toàn bộ tập dữ liệu:

$$\boldsymbol{\mu} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$$

với $\mathbf{x}_i$ là các vector đặc trưng của dữ liệu.

Bước 2 (Lặp)

Với mỗi bước lặp $t = 1, 2, \dots, T$ (với T là số lượng điểm cần chọn), thực hiện:

1. Chọn điểm tối ưu tiếp theo

- Điểm $\mathbf{x}_t$ được chọn từ tập các điểm chưa được chọn $\mathcal{X} \setminus \mathcal{S}_{t-1}$ sao cho:

$$\mathbf{x}_t = \arg \min_{\mathbf{x} \in \mathcal{X} \setminus \mathcal{S}_{t-1}} \text{dist} \left(\frac{\mathbf{x} + (t-1)\bar{\mathbf{x}}_{t-1}}{t}, \boldsymbol{\mu} \right)$$

- Trong đó, $\text{dist}(\mathbf{a}, \mathbf{b})$ là khoảng cách giữa hai vector có thể là euclidean hoặc cosine similarity $\mathbf{a}$ và $\mathbf{b}$.

2. Cập nhật trung bình tạm thời

$$\bar{\mathbf{x}}_t = \frac{\mathbf{x}_t + (t-1)\bar{\mathbf{x}}_{t-1}}{t}$$

với $\bar{\mathbf{x}}_1 = \mathbf{x}_1$.

3. Cập nhật tập điểm đã chọn

$$\mathcal{S}_t = \mathcal{S}_{t-1} \cup \{\mathbf{x}_t\}$$

Cải tiến thuật toán Herding có thể tập trung vào việc giải quyết những hạn chế hiện tại và mở rộng ứng dụng của nó. Có thể xem xét các hướng cải tiến sau đây.

2 Herding nhạy cảm với nhiễu

Vì các điểm nhiễu có thể được chọn nếu chúng có ảnh hưởng lớn trong không gian đặc trưng. Có thể cải tiến bằng cách thêm trọng số để giảm ảnh hưởng của các điểm dữ liệu có độ bất thường cao:

$$\mathbf{x}_t = \arg \min_{\mathbf{x} \in \mathcal{X} \setminus \mathcal{S}_{t-1}} \text{dist} \left(\frac{\mathbf{x} + (t-1)\bar{\mathbf{x}}_{t-1}}{t}, \boldsymbol{\mu} \right) w(\mathbf{x})$$

trong đó:

- $\mathcal{S}_{t-1}$ là tập các điểm đã được chọn
- $w(\mathbf{x})$ là trọng số dựa trên mức độ tin cậy của điểm dữ liệu

Xác định trọng số $w(\mathbf{x})$ dựa trên Isolation Forest

Trọng số được xác định thông qua các bước sau:

1. Sử dụng Isolation Forest để tính điểm bất thường cho mỗi điểm dữ liệu
2. Chuẩn hóa điểm bất thường về đoạn $[0,1]$:

$$\text{AnomalyScore}_{\text{norm}}(\mathbf{x}) = \frac{\text{AnomalyScore}(\mathbf{x}) - \min_{\mathbf{x}}}{\max_{\mathbf{x}} - \min_{\mathbf{x}}}$$

3. Tính trọng số:

$$w(\mathbf{x}) = \frac{1}{1 - \text{AnomalyScore}_{\text{norm}}(\mathbf{x})}$$

Ưu điểm của phương pháp này

- Hiệu quả: Isolation Forest có khả năng phát hiện bất thường trong không gian đa chiều, phù hợp với dữ liệu phức tạp.
- Đơn giản: Công thức trọng số trực quan và dễ triển khai.
- Giảm tác động của nhiễu: Các điểm có điểm số bất thường cao (nhiều) sẽ có trọng số thấp hơn, giảm khả năng được chọn.

Kiểm tra hiệu năng

Bảng 1: Kết quả kiểm tra hiệu suất

frac (%)	acc (%)	Dataset	Trạng thái
Traffic Sign			
50.00	99.15	Traffic Sign	Không dùng nhiều
40.00	99.25	Traffic Sign	Không dùng nhiều
30.00	98.37	Traffic Sign	Không dùng nhiều
20.00	96.68	Traffic Sign	Không dùng nhiều
10.00	93.86	Traffic Sign	Không dùng nhiều
50.00	99.61	Traffic Sign	Sử dụng nhiều
40.00	99.34	Traffic Sign	Sử dụng nhiều
30.00	99.34	Traffic Sign	Sử dụng nhiều
20.00	98.60	Traffic Sign	Sử dụng nhiều
10.00	96.19	Traffic Sign	Sử dụng nhiều
CIFAR-10			
100.00	82.68	CIFAR-10	Không dùng nhiều
50.00	68.82	CIFAR-10	Không dùng nhiều
40.00	68.13	CIFAR-10	Không dùng nhiều
30.00	62.46	CIFAR-10	Không dùng nhiều
20.00	56.94	CIFAR-10	Không dùng nhiều
10.00	52.34	CIFAR-10	Không dùng nhiều
50.00	68.14	CIFAR-10	Sử dụng nhiều
40.00	63.70	CIFAR-10	Sử dụng nhiều
30.00	64.66	CIFAR-10	Sử dụng nhiều
20.00	59.75	CIFAR-10	Sử dụng nhiều
10.00	63.70	CIFAR-10	Sử dụng nhiều

3 Hướng tiếp cận kết hợp Herding và Convex Hull

Kết hợp Herding với Convex Hull là một cách tiếp cận hiệu quả nhằm tối ưu hóa quá trình chọn tập con dữ liệu (coreset). Phương pháp này tận dụng ưu điểm của cả hai: Herding tập trung vào các điểm có ảnh hưởng lớn trong không gian đặc trưng, trong khi Convex Hull bảo toàn các điểm biên quan trọng.

Các bước thực hiện

1. Lọc không gian bằng Convex Hull trước Herding

- Mục tiêu: Giảm kích thước không gian dữ liệu ban đầu bằng cách chỉ giữ lại các điểm nằm trên biên của Convex Hull.
- Thực hiện:

1. Xây dựng Convex Hull từ tập dữ liệu $\mathcal{X}$.
 2. Giữ lại các điểm $\mathcal{X}_{\text{hull}} \subset \mathcal{X}$ nằm trên biên Convex Hull.
- Lợi ích: Loại bỏ các điểm nằm sâu trong không gian, giảm số lượng điểm đầu vào cho Herding.

2. Áp dụng Herding trên các điểm Convex Hull

- Mục tiêu: Chọn các điểm quan trọng nhất từ $\mathcal{X}_{\text{hull}}$ dựa trên ảnh hưởng trong không gian đặc trưng.
- Hàm mục tiêu Herding:

$$\mathbf{x}_t = \arg \max_{\mathbf{x} \in \mathcal{X}_{\text{hull}}} \langle \mathbf{x}, \mathbf{w}_t \rangle$$

- Cập nhật trọng số:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \mathbf{x}_t - \bar{\mathbf{x}}$$

trong đó $\bar{\mathbf{x}}$ là trung bình của toàn bộ tập dữ liệu ban đầu $\mathcal{X}$.

3. Kết quả cuối cùng

Sau khi thực hiện T bước lặp Herding trên các điểm $\mathcal{X}_{\text{hull}}$, tập hợp $\mathcal{C} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T\}$ chính là coreset đại diện cho dữ liệu ban đầu.

Ưu và nhược điểm của phương pháp kết hợp

- Ưu điểm:
 - Tăng tốc độ tính toán nhờ giảm kích thước không gian bằng Convex Hull.
 - Bảo toàn các đặc điểm biên quan trọng trong dữ liệu.
 - Tận dụng ưu điểm của Herding để chọn các điểm có ảnh hưởng lớn.
- Nhược điểm:
 - Việc xây dựng Convex Hull có thể tốn kém với dữ liệu lớn hoặc trong không gian chiều cao.
 - Có thể bỏ qua các điểm bên trong có ý nghĩa quan trọng.

4 Cải tiến Herding với học tăng cường (Reinforcement Learning) kết hợp với nhiễu

Học tăng cường (Reinforcement Learning - RL) có thể được áp dụng để cải tiến thuật toán Herding, giúp tối ưu hóa quá trình chọn coreset bằng cách tìm ra chiến lược chọn điểm hiệu quả nhất qua việc học từ phản hồi của môi trường.

4.1 Ý tưởng chính

Thay vì chọn điểm dữ liệu theo cách tham lam (greedy) như trong Herding truyền thống, chúng ta xây dựng bài toán chọn điểm như một bài toán học tăng cường:

- Mục tiêu: Tối ưu hóa chiến lược chọn điểm để giảm lỗi mô hình hoặc tăng tính đại diện của coreset.
- Cách tiếp cận: Sử dụng thuật toán RL để học chính sách π , xác định điểm tiếp theo $\mathbf{x}_t$ được chọn tại mỗi bước, đồng thời có thể thêm nhiều như một hành động để mô phỏng các tình huống thực tế.

4.2 Thành phần của bài toán RL

- Trạng thái (s_t):
 - Gồm các thông tin hiện tại của tập dữ liệu, ví dụ: danh sách các điểm đã được chọn, vector trọng số $\mathbf{w}_t$, và trung bình dữ liệu $\bar{\mathbf{x}}$.
- Hành động (a_t):
 - Chọn điểm dữ liệu tiếp theo $\mathbf{x}_t$ từ tập $\mathcal{X}$ hoặc thêm nhiều vào quá trình chọn điểm.
- Phần thưởng (r_t):
 - Đánh giá chất lượng của điểm được chọn, ví dụ: giảm lỗi mô hình hoặc tăng tính đa dạng của coreset.
 - Công thức gợi ý:

$$r_t = -\text{Loss}(C_t) + \lambda \cdot \text{Diversity}(C_t)$$

trong đó:

- * $\text{Loss}(C_t)$: lỗi mô hình trên coreset C_t .
 - * $\text{Diversity}(C_t)$: độ đa dạng của coreset.
 - * λ : hệ số điều chỉnh giữa hai thành phần.
- Chính sách ($\pi(s_t)$):
 - Xác định điểm $\mathbf{x}_t$ tối ưu dựa trên trạng thái s_t và có thể bao gồm hành động thêm nhiều.
 - Có thể học bằng các thuật toán RL như Q-learning hoặc Proximal Policy Optimization (PPO).

4.3 Quy trình thực hiện

1. Khởi tạo: Bắt đầu với trạng thái ban đầu s_0 , chưa chọn bất kỳ điểm nào.
2. Lặp:
 - (a) Dựa trên chính sách $\pi(s_t)$, chọn điểm $\mathbf{x}_t$ hoặc thêm nhiều vào quá trình chọn điểm.
 - (b) Cập nhật tập coreset $C_t = C_{t-1} \cup \{\mathbf{x}_t\}$.
 - (c) Tính phần thưởng r_t dựa trên chất lượng của điểm được chọn.
 - (d) Cập nhật chính sách $\pi(s_t)$ bằng thuật toán RL.
3. Kết quả: Sau T bước, tập coreset C_T là tập hợp các điểm được chọn.

Ưu và nhược điểm

- Ưu điểm:
 - Học được chiến lược chọn điểm tối ưu dựa trên phản hồi từ dữ liệu, bao gồm cả việc xử lý nhiễu.
 - Tăng khả năng tổng quát hóa và tính đa dạng của coreset.
- Nhược điểm:
 - Yêu cầu thời gian tính toán cao hơn so với Herding truyền thống.
 - Phụ thuộc vào thiết kế phần thưởng và thuật toán RL được chọn.

Tài liệu

- [1] J. L. Bentley, F. P. Preparata, and M. G. Faust, Approximation algorithms for convex hulls, *Commun. ACM*, **25** (1982), pp. 64–68.
- [2] N.-D. Hoang, N. K. Linh, and H. X. Phu, Inner approximation of the convex hull of finite sets. To appear.